CodeLab: The Complete Beginner's Guide

Welcome to **CodeLab**, a neo-brutalist "Hacker Console" platform! If you are a beginner looking at this codebase, navigating through modern web technologies might feel overwhelming. This guide is designed to break down the entire project—what each part does, how they connect, and what the code's real-world use is.

🏗️ 1. The Big Picture: What is CodeLab?

CodeLab is an online coding platform, similar to LeetCode or HackerRank. It allows users to:

1. Create an account and log in.
2. View a list of programming challenges.
3. Write code directly in the browser (using a built-in code editor).
4. Run code and see the output immediately.
5. Check their ranking on a global leaderboard.

🎓 Academic Context

This project was developed as a Full Stack Development (FSD) Lab Micro Project by a team of student developers (Mohammed Junade Pasha, Rishikesh Mepperi, Teja Reddy, Ishan Sai, Sathyanarayan, and Samiksha), under the guidance of **Mr. Shiva Kumar** as the Faculty Advisor.

🛠️ 2. The Technology Stack

Think of the project as a restaurant:

- **Frontend (Angular 21 & Tailwind CSS)**: This is the dining area and the menu. It's what the user sees and interacts with. Tailwind CSS provides the cool "neo-brutalist" (bold borders, high contrast) visual style.
- **Backend Database (Firebase)**: This is the freezer. It stores all the long-term data like user accounts, the list of coding problems, and past submissions.
- **Execution Engine (Express JS / API & Vercel)**: This is the kitchen. When you hit "Run Code", the code is sent via the API to the kitchen (`onlinecompiler.io` via `api/run-code.js`), which processes it and hands the result back.
- **IDE Engine (Monaco Editor)**: The sleek text box where users write code. It's actually the exact same core engine that powers **Visual Studio Code**!

📂 3. How the Code is Organized (Folder Structure)

All the frontend magic happens inside the `src/app/` folder. Here's a breakdown of the key folders:

`src/app/components/` (The "Pages" of the Website)

Components are individual building blocks of the user interface.

- `home` : The landing page you see before logging in.
- `login` & `signup` : The pages where users authenticate themselves.
- `problem-list` : The page that displays all available coding challenges in a list.
- `problem-detail` : The heavy lifter. This is the actual coding environment page, featuring the problem description on one side and the Monaco code editor on the other.
- `leaderboard` : A page that lists top users based on their scores.
- `navbar` & `sidebar` : The navigation menus that stay on screen as you move around.

`src/app/services/` (The "Brain" of the App)

If Components are what things *look like*, Services are what things *do*.

- `auth.service.ts` : Handles talking to Firebase to log people in and out securely.
- `problem.service.ts` : Reaches into the Firebase database to fetch the list of challenges and their hidden test cases.
- `submission.service.ts` : Saves a user's successful code runs into the database so they get credit for it.
- `leaderboard.service.ts` : Calculates user scores and fetches ranking data for the UI.
- `code-execution.service.ts` : Takes the code typed by the user and sends it to your `/api/run-code.js` backend to be executed.

`/api/` (The Serverless Backend)

- `run-code.js` : This is a Node.js API endpoint. Directly exposing the `onlinecompiler.io` secret API Key to the frontend is dangerous (hackers could steal it). Instead, when a user clicks "Run", the frontend sends the code here. This file safely holds the API key, talks to `onlinecompiler.io` , waits for the output, and sends the output back to the user securely.

`app.routes.ts` (The Map)

This file is the GPS of your application. When a user navigates to `yourwebsite.com/problems` , this file steps in and says: *"Ah, they want `/problems` . Render the `ProblemListComponent` on the screen!"*

⚙️ 4. The Data Flow (Step-by-Step Scenario)

Let's walk through what the code *actually* does when a user tries to solve a problem:

1. **User lands on CodeLab** (`app.routes.ts` routes to `HomeComponent`).
2. **User logs in** (`LoginComponent` captures the email/password, passes it to `AuthService` , which validates it with Firebase).
3. **User goes to Problems Page** (`ProblemListComponent` asks `ProblemService` for an array of all active coding challenges. Firebase returns the data, and the HTML loop `*ngFor` draws them as cards on screen).
4. **User clicks a Problem** (`ProblemDetailComponent` loads. It initializes the Monaco Editor so it looks like VS Code).
5. **User clicks "Run Code"**:
   - The user's code and selected language (e.g., Python or Java) are grabbed from the `Monaco Editor` .
   - `CodeExecutionService` takes this data and does an HTTP POST request to `/api/run-code` .
   - `api/run-code.js` grabs the secret API Key `99b25db1595628362bb49b0a5fa60f2a` and sends the text to `onlinecompiler.io` .
   - The compiler runs the code and returns "Hello World!".
   - `api/run-code.js` hands "Hello World!" back to `CodeExecutionService` .
   - `ProblemDetailComponent` displays the result in the console window.
6. **User passes all tests**: `SubmissionService` saves the timestamp and code into Firebase. The `LeaderboardService` recalculates the user's score to bump them up!

🎓 5. Summary

CodeLab is a beautifully connected modern web app. The **Angular Components** create a fast, dynamic user interface. The **Angular Services** cleanly separate the business logic (fetching data, submitting code). The **Firebase**

Database acts as the secure, real-time memory of the entire app. Finally, the **Serverless API (`api/run-code.js`)** securely bridges the gap between your users and a remote code compiler without exposing your secrets.

By reading through `app.routes.ts` , you can see every page that exists. By checking the `src/app/services` folder, you can see every action the app can take. By examining the `components` folder, you can see the visual HTML/CSS for every feature. Happy coding!